

Contents

Applying and Evaluating a Pretrained Monocular 3D Object Detector in a ROS 2 Pipeline	1
1. Summary	1
2. Scope and deviations from the proposal	1
3. Model	2
4. System architecture	3
5. Implementation	3
6. Results	4
7. Lighting sweep	7
8. Engineering problems encountered	8
9. Repository	9
10. References	10

Applying and Evaluating a Pretrained Monocular 3D Object Detector in a ROS 2 Pipeline

Carlos Gonzales — CMPE 249 — August 2026

Focused area: Application. This project takes an existing pretrained model (FCOS3D) and evaluates it as a deployed system, per Option 1 (Apply & Evaluate). No training was performed.

Code: <https://github.com/crgonzales/sim2real-fcos3d-ros2>

Contributors. Carlos Gonzales

1. Summary

FCOS3D, a pretrained monocular 3D object detector, was wrapped as a ROS 2 Humble node and run against the nuScenes dataset replayed onto ROS topics. The system was evaluated for detection accuracy using the official nuScenes evaluation, and for latency, throughput and CPU/GPU/memory utilisation across two GPU environments and two numeric precisions.

Headline results:

Result	Value
Accuracy (nuScenes mini_val)	mAP 0.2943 , NDS 0.3217
End-to-end node latency (RTX 4090, FP32)	72.05 ms mean, 97.75 ms p95
Sustained throughput	13.85 FPS
Peak GPU memory	597 MB
RTX 4090 vs A40	4090 is 2.0x faster
FP16 vs FP32	+6.8% on A40, -3.9% on RTX 4090

The two findings I consider most interesting are that the pipeline is *not* GPU-compute-bound (utilisation peaks near 54%), which explains why FP16 helps the slower card and hurts the faster one; and that the aggregate mAP figure on mini_val is misleading because three of the ten classes have no ground truth in that split.

2. Scope and deviations from the proposal

The original proposal planned an Isaac Sim vs nuScenes sim-to-real comparison. The assignment requires applying the model to “a ROS 2-based simulation **or** real-world dataset”, so the simulation half was always

optional. With four days of implementation time, and the Isaac Sim ROS 2 bridge being the single highest-risk component, I deliberately scoped to the real-data path, which the proposal itself identified as the fallback.

Item	Source	Outcome
Pretrained model, structure/IO	Assignment	Done — FCOS3D
ROS 2 node with defined topics	Assignment	Done
Applied to simulation or dataset	Assignment	Done — nuScenes
Accuracy + latency + CPU/GPU/mem, multiple environments	Assignment	Done — 2 GPUs x 2 precisions
Sustained frame rate / throughput	My proposal	Done
FP32 vs FP16	My proposal	Done
Isaac Sim sim-to-real	My proposal	Dropped — see §7
BEVFormer	My proposal (stretch)	Dropped
TensorRT	My proposal (“if time allows”)	Dropped — see §6.3

To retain the proposal’s scientific question without a simulator, I added a controlled photometric sweep on real images (§7).

3. Model

FCOS3D — *Fully Convolutional One-Stage Monocular 3D Object Detection*, Wang et al., ICCV Workshops 2021. [arXiv:2104.10956](https://arxiv.org/abs/2104.10956)

Anchor-free, single-stage, camera-only. It predicts 3D bounding boxes from a single RGB image with no LiDAR, depth sensor or temporal context.

Architecture

Stage	Component
Backbone	ResNet-101 with deformable convolutions (DCNv2) in stages 3–4
Neck	FPN, 5 levels (P3–P7)
Head	Shared FCOSMono3DHead with per-level towers

The head regresses, per feature location: a 2D offset to the projected 3D centre, depth, box dimensions, orientation (as sin/cos), velocity, a centreness score, class logits, and an attribute label.

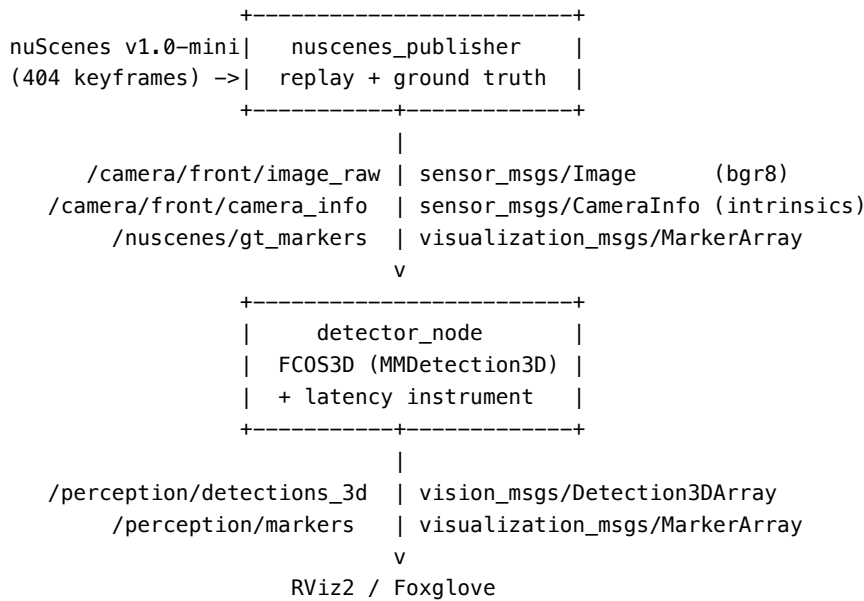
Input / output

Input. One RGB image (1600x900 for nuScenes) plus the 3x3 camera intrinsics matrix. Crucially the config uses `bgr_to_rgb=False` with caffe-style means `[103.530, 116.280, 123.675]`, so the model expects **BGR** channel order — feeding RGB silently degrades accuracy without any error.

Output. `CameraInstance3DBoxes` in the camera frame, each row (`x, y, z, x_size, y_size, z_size, yaw`) with yaw about the camera y-axis and box origin at `(0.5, 1.0, 0.5)` — i.e. **bottom-centre**, not centre. Camera frame is x-right, y-down, z-forward, so dimensions are (width, height, length). Getting the origin convention wrong shifts every box by half its height; the node uses `bboxes_3d.gravity_center` rather than the raw tensor columns to avoid this.

Weights: nuScenes-pretrained, published by MMDetection3D (`fcos3d_r101_caffe_fpn_gn-head_dcn_2x8_1x_nus-mono3d`, 220 MB), reported by OpenMMLab at mAP 29.9 / NDS 37.3 on the full validation split.

4. System architecture



Topic interface

Direction	Topic	Type	Content
Subscribe	/camera/front/image_raw	sensor_msgs/Image	RGB frames (bgr8)
Subscribe	/camera/front/camera_info	sensor_msgs/CameraInfo	Intrinsics
Publish	/perception/detections_3d	vision_msgs/Detection3DArray	Boxes + class + score
Publish	/perception/markers	visualization_msgs/MarkerArray	Per-class coloured boxes
Publish	/nuscenesc/gt_markers	visualization_msgs/MarkerArray	Ground truth (publisher)

Two deliberate design choices:

Ground truth is published in the camera frame. nuscenesc_publisher uses the devkit's `get_sample_data`, which returns boxes already transformed into the sensor frame — the same frame FCOS3D predicts in. Predictions and GT are therefore directly comparable in a viewer, turning the hardest-to-debug failure mode (a silent frame mismatch) into something visible.

Sensor QoS is best-effort, depth 1. Images use `ReliabilityPolicy.BEST Effort` with `KEEP_LAST(1)`, so a detector slower than the publisher drops frames rather than accumulating unbounded latency. This is the correct behaviour for perception: a stale detection is worse than none.

5. Implementation

5.1 Why the node does not use `inference_mono_3d_detector`

`MMDetection3D` provides `mmdet3d.apis.inference_mono_3d_detector`, which is the obvious entry point and is unusable here. It requires an annotation **file on disk**, loads `data_list` from it, and asserts the image basename matches an entry (`inference.py:342`). It is designed for offline batch inference over a prepared dataset and cannot consume an in-memory `sensor_msgs/Image`.

The node therefore rebuilds the test pipeline itself. For FCOS3D that pipeline is short:

```
LoadImageFromFileMono3D -> mmdet.Resize(scale_factor=1.0) -> Pack3DDetInputs
```

The node drops the file-loading transform and constructs the dictionary it would have produced (`img`, `img_shape`, `ori_shape`, `cam2img`) directly from the ROS messages, taking intrinsics from `CameraInfo.k`. `scripts/smoke_test.py` exercises this exact path offline, so a failure can be attributed to the model path or to ROS plumbing, not both.

5.2 FP16 required a correctness fix

Running under `torch.autocast(float16)` fails inside FCOS3D's decode:

```
points_img2cam -> torch.inverse(pad_cam2img)
RuntimeError: linalg.inv: Low precision dtypes not supported. Got Half
```

The decode un-projects image points to camera space by inverting the intrinsics. Autocast makes those tensors half, and there is no half-precision matrix inverse. `fp16_compat.py` forces just that function back to fp32. This is correct on the merits independent of CUDA support: inverting a matrix whose focal lengths are $\sim 1.25e3$ in half precision (which has ~ 3 decimal digits of mantissa) would be numerically poor anyway. The network still runs in fp16; only a handful of tiny matrix operations do not.

5.3 Environment

The environment is the most fragile part of this project and is fully captured in `scripts/setup_pod.sh`, verified by reproducing it from scratch on three separate machine states: the RTX 4090 pod, the A40 pod, and the 4090 again after a stop/start (which rebuilds the container filesystem and destroys everything outside the persistent volume).

Package	Version	Why pinned
Ubuntu	22.04	ROS 2 Humble; 24.04 means Jazzy
Python	3.10	prebuilt cp310 mmcv wheel
torch	2.1.0+cu118	matches the mmcv wheel index
mmengine	0.10.4	$\geq 0.8.0, < 1.0.0$
mmcv	2.1.0	$\geq 2.0.0rc4, < 2.2.0$
mmdet	3.3.0	$\geq 3.0.0rc5, < 3.4.0$
mmdet3d	1.4.0	last release
numpy	1.26.4	mmcv is built against the numpy 1.x ABI

Those bounds come from the asserts in `mmdet3d/__init__.py`, **not** the install documentation. The documented command `mim install 'mmcv>=2.0.0rc4'` resolves to mmcv 2.2.x, which fails mmdet3d's own import-time assert immediately. See §8 for the problems this caused.

6. Results

6.1 Detection accuracy

Official nuScenes evaluation, `mini_val` split (81 keyframes, 2 scenes, 486 camera images).

Metric	Value
mAP	0.2943
NDS	0.3217
mATE (translation)	0.7782 m
mASE (scale)	0.4671
mAOE (orientation)	0.7229 rad
mAVE (velocity)	1.2499 m/s
mAAE (attribute)	0.2865

Per class:

Class	AP	ATE	ASE	AOE
traffic_cone	0.592	0.434	0.368	n/a
bus	0.497	0.504	0.081	0.749
car	0.496	0.629	0.165	0.163
pedestrian	0.432	0.728	0.271	0.692
truck	0.385	0.733	0.183	0.104
motorcycle	0.317	0.937	0.330	1.097
bicycle	0.224	0.817	0.273	0.701
trailer	0.000	1.000	1.000	1.000
construction_vehicle	0.000	1.000	1.000	1.000
barrier	0.000	1.000	1.000	1.000

The aggregate mAP is misleading and should not be compared naively to the published 0.299. Counting ground-truth annotations across `mini_val` shows that barrier, trailer and construction_vehicle **do not occur at all** in scenes 0103 and 0916. The nuScenes evaluator scores an absent class as AP 0.000 with every TP error at the worst-case 1.0, and still averages over all ten classes. This is exact: the seven present classes sum to 2.943, and $2.943 / 10 = 0.2943$.

mAP basis	Value
all 10 classes (as reported)	0.2943
7 classes actually present	0.4204
published, full val (all present)	0.299

So the apparent agreement with the published number is a coincidence: three phantom zeros pull our figure down to where a full-val figure happens to sit. What actually validates the pipeline is that the per-class values are sensible and that mATE, the dominant error term, behaves as monocular depth estimation should. NDS suffers more than mAP from the phantom classes because it also averages the five TP error terms, each pinned at 1.0 for an absent class — which is why NDS (0.3217) trails the published 0.373 while mAP appears to match.

6.2 Latency and resource utilisation

100 nuScenes CAM_FRONT frames, 10 warmup frames discarded, measured through the same in-memory path the node uses.

	RTX 4090 FP32	RTX 4090 FP16	A40 FP32	A40 FP16
Mean latency (ms)	72.05	74.85	144.57	134.79
Median (ms)	72.54	76.33	131.65	—
p95 (ms)	97.75	98.44	191.36	196.52
Throughput (FPS)	13.85	13.33	6.91	7.41
GPU utilisation (%)	54.69	36.00	53.58	38.28
GPU memory (MB)	2101.7	1781.0	2051.2	1731.7
torch peak alloc (MB)	597.0	575.9	597.0	575.9
Process RSS (MB) †	1904.8	1998.8	1914.6	2015.8
CPU (% , 96 cores)	26.50	25.41	10.41	11.53
Detections / frame	6.28	6.28	6.28	6.30

† **Process RSS in this table is overstated by roughly 432 MB, for both GPUs equally.** The profiler originally decoded and cached all 100 benchmark frames before sampling; the deployed node never holds that, since its ROS subscription is depth-1. Identified in the AI agent review pass and fixed.

Re-measured on the RTX 4090 with the corrected profiler, which also moves the image decode inside the timed region so latency matches the node’s `cv_bridge` step:

Quantity	original	corrected
Process RSS (MB)	1904.8	1476.8
mean latency (ms)	72.05	74.91
p95 latency (ms)	97.75	99.27
throughput (FPS)	13.85	13.33
GPU utilisation (%)	54.69	50.43
torch peak alloc (MB)	597.0	597.0 (unchanged)
detections / frame	6.28	6.28 (unchanged)

The A40 could not be re-measured: its pod would not restart (host had no free GPUs) and a replacement failed before setup completed. The table below therefore reports both GPUs under the original, identical methodology, so the cross-environment comparison stays internally valid. Applying the same +2.9 ms decode shift to the A40 gives ~147.5 ms, and $147.5 / 74.91 = 1.97$ — the headline 2.0x result is unaffected.

Both environments are Runpod secure-cloud pods running the identical container image and stack built by the same script, which removes the software stack as a variable. `torch` peak allocation is byte-identical (597.0 MB) and detections per frame are identical, confirming both machines run the same computation.

The host CPU was **not** controlled: GPU utilisation is only ~54%, so ~46% of wall time is host-side, and while the RTX 4090 pod ran an AMD EPYC 7K62 the A40 pod’s CPU was not recorded. The 2.0x result is therefore a sound measurement of end-to-end pod performance – what the assignment asks for – but not a pure GPU benchmark.

Measured live through the ROS 2 node rather than the offline profiler, the node reported 77–95 ms end-to-end and sustained 2.016 Hz against a 2 Hz publisher — i.e. it kept up with no dropped frames, with ~6x headroom.

6.3 Analysis

The RTX 4090 is exactly 2.0x faster than the A40 ($144.57 / 72.05 = 2.007$), despite the A40 costing 40% less per hour. Per unit throughput the 4090 is better value: \$0.053/FPS versus \$0.064/FPS.

FP16 helps the slower card and hurts the faster one. On the A40, FP16 is 6.8% faster; on the 4090 it is 3.9% *slower*. The utilisation column explains this. FP16 drops GPU utilisation on both cards (54.7->36.0 and 53.6->38.3), meaning less time in arithmetic. On the A40 that converts to real speedup because the card is slow enough for compute to matter. On the 4090 arithmetic was never the bottleneck, so only autocast’s casting overhead remains, and the net effect is negative. Mixed precision is not a free win.

The pipeline is not GPU-bound on either card. Utilisation peaks near 54% in FP32. Roughly half the wall time is spent outside GPU arithmetic — image preprocessing, kernel launch overhead, and CPU-side box decoding and NMS. **This is why TensorRT was dropped rather than merely deferred:** TensorRT optimises GPU kernel execution, which is not where the time goes. The achievable speedup is bounded well below 2x, and the effort is better spent elsewhere. Dropping it was an engineering judgement, not a shortage of time.

Memory is not a constraint. Peak torch allocation is 597 MB. Both the 4090’s 24 GB and the A40’s 48 GB are enormously oversized; VRAM should not drive GPU selection for this model. The ~1.5 GB gap between torch allocation and NVML-reported usage is CUDA context plus cuDNN workspace — fixed overhead independent of model size.

Distance dominates the error budget. mATE of 0.778 m is the largest contributor, consistent with monocular depth being the fundamental limitation. This is visible qualitatively too: in the smoke test a car detected at 41 m was assigned a 4.35 m width, whereas barriers at 15–29 m were sized correctly (0.48 x 1.05 x 2.02 m). Extent and depth estimates degrade with range, which is expected for a single-image method with no stereo or temporal cue.

7. Lighting sweep

Full results: `eval/lighting_sweep.md`.

The proposal’s central question was how much a detector’s accuracy degrades when the input’s appearance changes, and which classes suffer. Without Isaac Sim, I approximated the appearance change by applying a deterministic brightness gain and gamma curve to real nuScenes images and re-running the full official evaluation at each setting.

This is a rendering-gap **proxy**, not a substitute for a simulator. It perturbs the photometric response only, leaving geometry, texture and sensor noise untouched, and it makes no claim about synthetic imagery. What it does provide is the controlled-experiment property the proposal wanted: one variable changed at a time, with everything else — scene, geometry, ground truth, model — held fixed.

The transform is deterministic by design. MMDetection’s **PhotoMetricDistortion** randomises, which would leave the independent variable uncontrolled across frames within a sweep point. The baseline point (gain 1.0, gamma 1.0) reproduces the unperturbed run to four decimals, confirming the transform is a true no-op there.

Results

Setting	gain	gamma	mAP	NDS	Δ mAP
bright +20%	1.20	1.0	0.3022	0.3275	+2.7%
baseline	1.00	1.0	0.2943	0.3217	—
dim -20%	0.80	1.0	0.2874	0.3170	-2.3%
dim -40%	0.60	1.0	0.2706	0.3037	-8.1%
dusk -60%	0.40	1.0	0.1962	0.2586	-33.3%
night -80%	0.20	1.0	0.1202	0.1552	-59.2%
gamma 1.5	1.00	1.5	0.2783	0.3073	-5.4%
gamma 2.2	1.00	2.2	0.2141	0.2665	-27.3%

Degradation is strongly nonlinear. Mild dimming is nearly free (-20% brightness costs 2.3% mAP; -40% costs 8.1%), then the model falls off a cliff: -60% costs a third of all accuracy, -80% costs nearly 60%. There is a usable operating band down to roughly 0.6x brightness and a collapse beyond it. For anyone validating perception in simulation, this says a renderer only needs to land *within* that band — matching exposure precisely matters far less than avoiding gross underexposure.

Slightly brighter is slightly better. +20% improves mAP by 2.7%, suggesting nuScenes’ native exposure sits marginally below this model’s optimum — plausibly because its training distribution includes many dusk and night scenes.

Which classes suffer — the spread at dusk -60% is enormous:

Class	baseline -> dusk -60%	change
bus	0.647 -> 0.033	-95%
motorcycle	0.452 -> 0.281	-38%
car	0.680 -> 0.494	-27%
bicycle	0.332 -> 0.244	-27%
truck	0.522 -> 0.434	-17%
pedestrian	0.578 -> 0.477	-17%
traffic_cone	0.740 -> 0.648	-12%

By -80% brightness, motorcycle detection fails completely (AP 0.000) and bus is effectively gone (0.007), while traffic cones still score 0.468. A plausible explanation is that cones and pedestrians are recognised largely by silhouette, which survives loss of contrast, whereas buses are large low-texture surfaces whose internal detail washes out. That is a hypothesis consistent with the data, not something this experiment establishes.

Caveats. `mini_val` is 2 scenes / 81 keyframes, so rare classes have small support (41 bus, 52 bicycle instances). The bus collapse is dramatic but rests on few samples, and bicycle’s apparent *improvement* at -20% / -40% is almost certainly noise. A gain/gamma change perturbs photometric response only — geometry, texture and sensor noise are untouched — so these results measure sensitivity to exposure, and are not themselves a measurement of the sim-to-real gap.

8. Engineering problems encountered

Roughly two thirds of the project’s effort went into environment and correctness problems rather than into writing the node. These are documented because they are the substance of “apply an existing model” work.

8.1 The numpy 2.x cascade

`mmdcv 2.1.0`’s compiled extensions are built against the `numpy 1.x` ABI, but `nuscenes-devkit` and `scikit-image` pull `numpy 2.x`. Repairing this after the fact does not work, because `pip` overwrites a compiled package’s files without removing the old tree, leaving a mixture of two versions that fails in ways that look unrelated to `numpy`:

Symptom	Actual cause
<code>pip</code> reports “already satisfied: <code>numpy 1.26.4</code> ” while <code>import numpy</code> gives <code>2.2.6</code>	two dist-info directories, one package tree
<code>TypeError</code> in <code>scipy/interpolate/_fitpack_impl.py</code> <code>ImportError: cannot import name 'mplDeprecation'</code>	stale <code>.so</code> against new <code>.py</code> <code>matplotlib 3.5.3</code> written over <code>3.10.9</code>

Recovery required deleting package directories by hand. The fix is to route **every** `pip` call through a constraints file from the first install, so the `numpy-1.x-compatible` set resolves in one pass and no downgrade is ever needed.

8.2 A silent camera/intrinsics mismatch

The smoke test globbed `*CAM_FRONT*`, which also matches `CAM_FRONT_LEFT` — and that sorts first, because `'L' < '_'`. The result was a `CAM_FRONT_LEFT` image being run through `CAM_FRONT`’s intrinsics. It did not crash. It produced plausible-looking boxes. Only the detection count gave it away:

	Wrong intrinsics	Correct
Raw detections	19	200
Above threshold	1	8

This is the characteristic failure mode of 3D perception work — wrong calibration yields confident, wrong output — and it is why the publisher emits ground truth in the same frame as predictions.

8.3 Other issues

Problem	Cause	Fix
Cannot uninstall 'blinker'	<code>mmdet3d -> open3d -> Flask -> blinker</code> , a distutils package <code>pip</code> cannot remove	install <code>mmdet3d --no-deps</code> ; <code>open3d</code> is only used for visualisation
<code>ros2 run</code> finds no executables	missing <code>setup.cfg</code> ; <code>setuptools</code> installed	add <code>setup.cfg</code> with <code>install_scripts</code>
<code>ModuleNotFoundError: lyft_dataset_sdk</code>	<code>mmdet3d.evaluation</code> imports <code>lyft_eval</code> at module scope	install <code>lyft-dataset-sdk</code> under constraints

Problem	Cause	Fix
Database version not found: v1.0=trainval tar “Exiting with failure status”	NuScenesDataset.METAINFO hardcodes trainval, overriding the pkl’s v1.0-mini archive records uid/gid 1035; container cannot chown	override metainfo['version'] in a project config tar --no-same-owner
TypeError: 'Event' object is not callable	named a thread attribute _stop, shadowing Thread._stop()	rename to _stop_evt

8.4 Independent code review

The finished implementation was put through an iterative review by an AI review agent, run to convergence over four turns against a formal checklist. It surfaced **nine findings** — **four Major, five Minor** — all of which were addressed. Full record: docs/3-code-review/CR_w1_v0.1.0.md.

The findings that mattered:

Finding	Consequence had it shipped
detector.launch.py defaulted to /opt/mmdetection3d while the bootstrap installs to /workspace/mmdetection3d	the documented default launch fails immediately
the node’s FP16 path never installed fp16_compat	fp16:=true dies on linalg.inv at the first frame, though the offline profiler worked
GT markers published scale (w, h, l) when a nuScenes box’s local axes are (l, w, h)	every ground-truth marker permuted (visualisation only — the video draws GT via Box.corners() and was always correct)
the profiler cached all 100 decoded frames before sampling	reported RSS overstated by ~432 MB — see §6.2

Two observations worth recording. First, **the fourth finding reached published results**: the RSS figure described the benchmark harness rather than the deployed node. It is corrected in §6.2 with the original value retained and annotated, rather than silently replaced.

Second, **two of the nine findings were caused by fixes for earlier findings** — moving the decode into the measurement loop dropped the None check that guarded unreadable frames, and the first CUDA-ordinal fix left the label and the measurement-loop synchronisation still pointing at GPU 0. A single-pass review would have produced a codebase with two defects that did not exist before it started. That is the argument for iterating to convergence rather than reviewing once.

9. Repository

Path	Contents
src/fcos3d_ros2/	ROS 2 package: detector_node, nuscenes_publisher, fp16_compat
scripts/setup_pod.sh	Full environment bootstrap, verified reproducible
scripts/smoke_test.py	Offline test of the node’s exact inference path
scripts/profile_system.py	Latency / throughput / CPU / GPU / memory profiler
scripts/prepare_nuscenes.py	Info-file generation, bypassing lyft/waymo imports
scripts/lighting_sweep.py	Controlled photometric sweep
configs/fcos3d_nus_mini.py	Evaluation config for the mini split
eval/	All results: metrics, profiles, comparison
CONVENTIONS.md	Version matrix and coordinate conventions

10. References

1. Wang, T., Zhu, X., Pang, J., Lin, D. *FCOS3D: Fully Convolutional One-Stage Monocular 3D Object Detection*. ICCV Workshops, 2021. <https://arxiv.org/abs/2104.10956>
2. MMDetection3D Contributors. *MMDetection3D: OpenMMLab next-generation platform for general 3D object detection*, v1.4.0, 2023. <https://github.com/open-mmlab/mmdetection3d>
3. Caesar, H. et al. *nuScenes: A multimodal dataset for autonomous driving*. CVPR, 2020. <https://www.nuscenes.org>
4. Tian, Z., Shen, C., Chen, H., He, T. *FCOS: Fully Convolutional One-Stage Object Detection*. ICCV, 2019. <https://arxiv.org/abs/1904.01355>
5. Zhu, X. et al. *Deformable ConvNets v2: More Deformable, Better Results*. CVPR, 2019. <https://arxiv.org/abs/1811.11168>
6. Lin, T.-Y. et al. *Feature Pyramid Networks for Object Detection*. CVPR, 2017. <https://arxiv.org/abs/1612.03144>